LifeStation Caregiver Mobile App v1.0 — Proposal

Submitted to: LifeStation
RFP: LifeStation Caregiver Mobile App – v1.0 Development
Primary Contact: Dan Entin, VP Product & Technology

Submitted by: AIAssistant.co
Point of Contact: [Vaidhi: Please provide your Name, Title, Email, and Phone here]
Date: March 2026

1. Company Overview

AIAssistant.co is a Gen AI technology company based in Silicon Valley that provides AI-powered assistants automating phone calls, SMS messaging, and web chat for businesses. We specialize in creating intelligent virtual assistants that handle customer communications 24/7, with proven expertise in healthcare, senior care, hospitality, and enterprise communications.

Our team combines decades of experience in AI, machine learning, and business automation, with successful deployments across thousands of companies. We help businesses improve customer satisfaction while reducing operational costs by up to 80% through intelligent automation.

Key Company Highlights

- Silicon Valley-based AI technology company
- Expertise in healthcare and HIPAA-compliant communications
- 24/7 AI assistant deployment in under 5 minutes
- Proven ROI of 10–50x cost savings vs human staff
- Advanced Large Language Models (LLMs) with sub-2 second response times
- Trusted by leading companies including healthcare and senior care providers

2. Executive Summary

LifeStation is launching its first caregiver-facing mobile application to close competitive gaps in caregiver experience, enable customer self-service, and establish a scalable digital foundation for LSaaS and future smartphone-based safety offerings.

We propose a low-risk, accelerated delivery approach: build v1.0 leveraging our proven experience with LifeStation/Brighton React Native and backend development against the Affiliated Monitoring APIs (Account API, REST Report API, Brighton Device API). We will close the remaining gaps required by the RFP—UX polish, billing (D2C), HIPAA/security hardening, analytics and observability, and web delivery—to deliver a production-ready, caregiver-first experience.

Why this approach wins

- **Proven foundation:** Extensive experience with LifeStation caregiver app and monitoring stack
- **Speed to value:** Focus on UX polish and RFP completion vs rebuilding
- **Lower risk:** Existing integrations reduce uncertainty
- **Future-ready:** Modular architecture supports future roadmap

Outcome

- Caregiver-first mobile experience (iOS & Android)
- Web experience aligned to v1.0 scope
- Scalable platform for future expansion

3. Why AIAssistant.co is the Ideal Partner

Proven Healthcare & Senior Care Expertise

- Specialized experience in healthcare and senior care
- HIPAA-compliant deployments
- Deep understanding of caregiver workflows
- Existing integrations with healthcare providers

Advanced AI Technology Stack

- Cutting-edge LLMs optimized for healthcare
- Sub-2 second response times
- Natural language conversations
- 24/7 automation

Scalable Business Automation

- 10–50x cost reduction
- Unlimited concurrency
- No downtime
- 5-minute deployment

Enterprise-Grade Security & Compliance

- HIPAA-compliant architecture
- Audit logging and reporting
- Private cloud / on-premise options

Measurable Business Impact

- Proven ROI
- High satisfaction rates
- Strong automation outcomes

Technical Integration Capabilities

- API-first architecture
- CRM + healthcare integrations
- Real-time synchronization
- Multi-channel automation

4. Added Features - Our Unique Advantages

A. GPS and Geofencing

- Interactive geofence setup with Google Places
- Real-time tracking with history
- Entry/exit alerts

- Custom radius (0–10 km)

B. In-App Calling

- One-touch calling
- Contact integration
- Role-based permissions

C. Medication Reminders

- Full medication tracking
- Smart push notifications
- Multi-dosage support
- Custom scheduling

5. Enhanced User Roles

Standard Roles

- Caregivers
- Seniors

Additional Roles

Admin

- Full system control
- User + device management
- Reporting and compliance

Family Members

- View-only access
- Notifications
- Emergency escalation

Role Matrix

- **Senior:** Profile, help requests
- **Caregiver:** Full monitoring
- **Family:** Limited access

- **Admin:** Full control

6. Dependencies - Implementation Coordination

Critical Dependencies (Week 1–2)

- **Development Environment Access:** AWS development credentials
- **API Access:** Development/staging credentials for Affiliated Monitoring APIs (Account, Device, Reports)
- **Functional Specifications:** Finalized detailed functional specs (noted as "in progress" in RFP)
- **HIPAA Compliance Framework:** LifeStation's specific HIPAA requirements and audit checklist

Important Coordination (Week 3–4)

- **Firebase Project Setup:** Firebase configuration for push notifications (FCM)
- **App Store Accounts:** Access to LifeStation's Apple Developer and Google Play Console accounts
- **Analytics Configuration:** Integration setup for Mixpanel, Datadog, Crashlytics, Userpilot
- **UAT Process Definition:** User acceptance testing scenarios and approval workflows

Ongoing Coordination

- **Release Management:** Deployment pipeline approval process and production release coordination
- **Monitoring Setup:** Production monitoring thresholds and alerting configuration

7. Core Features Delivery (RFP 4.2)

Authentication

SMS-based verification + role-based access

Device Monitoring

Real-time status, alerts, history

Emergency Contacts

Full CRUD + dispatch integration

Billing

Secure payment support

8. Understanding of Goals

Strategic Goals

- Caregiver experience
- Reduced cost-to-serve
- Scalable platform

v1.0 Scope

- Authentication
- Device monitoring
- Notifications
- Billing
- Web app

9. Delivery Approach

Workstreams

- UX & Design
- Mobile
- Web
- Security

- QA

Execution

- Agile (2-week sprints)
- Bi-weekly demos
- Incremental delivery

10. UX & Design Methodology

10.1. Collaborative UX Discovery

We will conduct intensive collaborative UX discovery sessions with LifeStation stakeholders to deeply understand caregiver workflows, senior user needs, and business objectives. Key activities include:

- Stakeholder workshops with **Dan Coppola's product team**.
- **User journey mapping** sessions to identify pain points and opportunities.
- **Competitive analysis** of Medical Guardian, Freeus, and Aloe Care apps to understand market expectations.
- **Persona development workshops** to create detailed profiles for caregivers and seniors.
- **Contextual interviews** with existing LifeStation customers to gather real-world insights.

This approach ensures that **design decisions are grounded in actual user needs and business requirements**, not assumptions.

10.2. Figma-Based Interactive Prototypes

We will create comprehensive Figma-based interactive prototypes that allow stakeholders to experience the full user journey before development. Our approach includes:

- **Interactive prototypes** with realistic data and navigation flows.
- **Responsive design mockups** demonstrating adaptability across screen sizes and orientations.

- **Component libraries** with reusable design elements for consistency.
- **Role-specific prototype flows** for caregivers, seniors, family members, and admins.
- **Realistic content and error states** to provide a complete user experience.

10.3. Structured User Testing Prior to Development

We will validate designs with real users through a rigorous testing program, including:

- **Moderated usability testing** with LifeStation customers from both caregiver and senior groups.
- **Task-based testing** on critical functions: emergency response, device monitoring, contact management.
- **Accessibility testing** with seniors using assistive technologies (screen readers).
- **A/B testing** of key interface elements for senior accessibility and caregiver efficiency.
- **Multiple testing rounds**: initial concept validation, detailed interaction testing, and final validation before development handoff.
- **Analysis of recorded sessions** to identify usability issues, confusion points, and improvement opportunities.

10.4. Design for Senior Accessibility & Caregiver Efficiency

We prioritize **senior accessibility** and **caregiver efficiency** using evidence-based design principles:

Senior Accessibility:

- Large touch targets (≥44px) for motor skill variations.
- Clear visual hierarchy with generous white space to reduce cognitive load.
- Simple, consistent navigation patterns that are easy to learn.
- Large, readable fonts (≥16pt) and minimal complex gestures.

Caregiver Efficiency:

- Dashboard views providing at-a-glance status for multiple seniors.
- Quick action buttons for tasks like calling emergency contacts or requesting device signals.
- Contextual information display showing the most relevant data based on device status and recent events.

10.5. Minimize Tap Count for Critical Functions

We optimize the interface to **reduce taps for the most critical actions**, ensuring emergency and monitoring features are immediately accessible:

- **Location visibility:** direct from main dashboard (1 tap max).
- **Emergency contact calling:** accessible within 2 taps from any screen.
- **Alarm and event history:** accessible within 2 taps from the home screen.
- **Device status information:** visible on the main dashboard without extra taps.
- **Contextual menus & smart shortcuts** that adapt based on user behavior and device status.
- Critical emergency functions have **dedicated quick-access buttons** visible across all screens.

11. Shared Architecture Strategy (React Native, Web & Code Reuse)

While the initiative is centered around React Native, we recommend a **pragmatic approach to code reuse**—maximizing shared logic where it adds value, while avoiding unnecessary complexity that can impact performance and maintainability.

Proposed Web Architecture

We propose building the web application using **React (TypeScript)** rather than relying heavily on React Native Web.

React Native Web, while useful in certain cases, introduces limitations in responsiveness, performance, and ecosystem support. For a scalable, production-grade web experience, **React is the more stable and efficient choice**.

Code Reuse Strategy

We will focus on **selective reuse (20-30%)**:

- **Business Logic (High Reuse – ~30–40%)**
  APIs, data models, validation, and shared services
- **UI Components (Limited Reuse – ~10–15%)**
  Only simple, platform-agnostic components
- **Architecture & Patterns (High Reuse)**
  Shared design systems, structure, and state management approaches

Shared Components

- API and service layer
- Authentication and session logic
- Validation schemas and utilities
- Design tokens (colors, spacing, typography)

Key Limitations of Deep Reuse

- Responsive styling is harder with React Native Web
- Limited library compatibility
- Performance overhead vs native React
- Increased debugging complexity
- Higher learning curve for developers

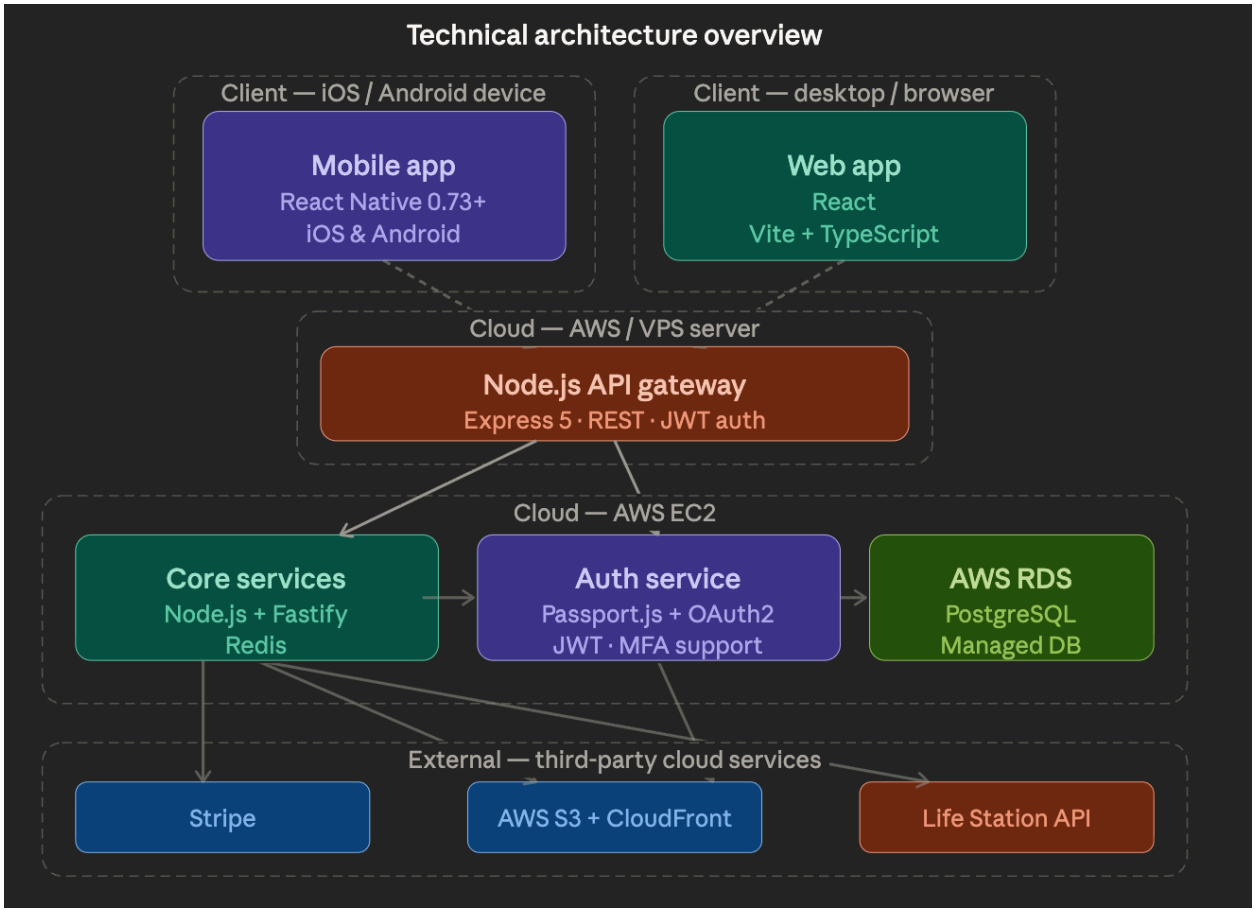
Conclusion

We will **maximize reuse where practical (logic, patterns)** while building a **dedicated React app** for performance and scalability.

With AI significantly accelerating development, strict code sharing is no longer a primary constraint. It is often more efficient to generate and tailor platform-specific code than to force reuse that could introduce complexity or performance trade-offs.

Our focus will be on shared architecture and consistency, while allowing flexibility for platform-specific optimizations. This ensures faster delivery, lower maintenance overhead, and a better overall user experience.

12. Technical Architecture



Mobile

Built using **React Native (TypeScript)** for iOS and Android with a shared codebase.

- State: **Redux Toolkit / Zustand**
- Navigation: **React Navigation**
- API: **Axios / React Query**
- Secure storage for sensitive data

Web

Built using **React (TypeScript)**.

- Framework: **Vite (SPA)**
- Styling: **Tailwind CSS / Styled Components**

- Data: **React Query**

Backend

Implemented in **Node.js (Fastify)** for high performance.

- APIs: **REST + WebSockets (real-time streaming)**
- DB: **PostgreSQL**
- Cache: **Redis**
- AI: Pluggable interfaces for **LLM, ASR, TTS**

Typescript (mobile, web & backend)

TypeScript will be used across the stack for end-to-end type safety and consistency.

- Shared types for APIs and models across mobile, web, and backend
- Strongly typed APIs with runtime validation
- Reusable business logic and utilities
- Better developer experience with safer refactoring and scalability

Integrations

- **Twilio** (sms): optional
- **AWS** (S3, RDS, CloudFront)
- Auth (JWT/OAuth)
- Stripe payment

Infrastructure

- **Nginx** as reverse proxy and load balancer
- Security: TLS, RBAC, logging, basic HIPAA safeguards

Third-Party Services & Tools

- **SMS Provider:** Twilio – SMS authentication and notifications
- **Payment Processing:** Stripe – secure credit card processing
- **Maps & Geolocation:** Google Maps Platform – location tracking and geofencing

Development & Project Management Tools

- **Code Repository:** GitHub – version control and collaboration
- **Project Management:** Jira – sprint planning, bug tracking, feature status
- **Communication:** Slack/Teams integration with Jira for real-time updates
- **Documentation:** Confluence or GitHub Wiki for technical documentation
- **Design & Prototyping**: Figma – UI/UX design, interactive prototypes, design system management

Quality Assurance & Testing

- **Mobile Testing:** Detox – React Native end-to-end testing
- **Web Testing:** Playwright – cross-browser automated testing
- **Performance Testing:** Lighthouse for web performance, React Native performance profiler

Analytics & Monitoring (LifeStation Specified)

- **Product Analytics:** Mixpanel – user behavior and feature usage tracking
- **Application Monitoring:** Datadog – performance monitoring and alerting
- **Crash Analytics:** Crashlytics – crash reporting and stability monitoring
- **User Onboarding:** Userpilot – in-app guidance and engagement

13. Security & HIPAA

We implement a practical HIPAA-aligned security model focused on protecting Protected Health Information (PHI) and sensitive patient data, ensuring baseline compliance for v1.0.

Core Principles

- Secure handling, storage, and transmission of PHI
- Strict, role-based access with full auditability
- Prevention of unauthorized access, leakage, or misuse
- Data minimization and privacy-first design

Security Controls

- **Encryption:** TLS 1.2+ in transit, encryption at rest
- **Access Control:** RBAC, least privilege, secure authentication (JWT + refresh tokens)
- **Infrastructure Security:** AWS VPC isolation, restricted network access, secure APIs, environment separation
- **Data Handling:** No PHI in logs or third-party tools, masking/redaction where applicable

14. Analytics & Observability

- Mixpanel
- Datadog
- Crashlytics
- Userpilot

15: QA & Testing Approach

Our QA approach ensures the LifeStation caregiver mobile application meets the **highest standards for reliability, performance, and user experience**. Given the life-critical nature of senior care apps, we implement a **multi-layered testing strategy** covering functionality, security, and accessibility.

Automated Testing Framework

- **Integration Testing:**

    - Automated validation of API endpoints, database operations, and third-party services (Twilio, Stripe, Google Maps)
- **End-to-End Testing:**
    - **Mobile:** Detox for React Native iOS and Android automated user journeys
    - **Web:** Playwright for cross-browser testing (Chrome, Safari, Firefox, Edge)
- **Component Testing:**
    - React Testing Library for UI component validation and interaction testing

Device Coverage & Compatibility Testing

Mobile Device Matrix

- **iOS:** iPhone 12–15 (iOS 15+) across different screen sizes and orientations
- **Android:** Samsung Galaxy S21+, Google Pixel 6+, OnePlus 9+ (Android 11+)
- **Tablet Support:** iPad (9th gen+), Samsung Galaxy Tab
- **Physical Device Testing:** Real devices via AWS Device Farm or BrowserStack

Web Browser Compatibility

- **Browsers:** Chrome 90+, Safari 14+, Firefox 88+, Edge 90+
- **Responsive Design Testing:** Desktop (1920×1080), Tablet (768×1024), Mobile (375×667)

Accessibility Testing (Critical for Senior Users)

- **Screen Reader Compatibility**
- **Touch Target Size:** Minimum 44px for senior accessibility
- **Font Size & Readability:** System font scaling up to 200% for vision-impaired users

Performance Testing

- **App Startup Time:** <3 seconds cold start
- **Memory Usage:** Optimized to prevent leaks and ensure smooth operation on older devices
- **Network Performance:** Testing under 3G, 4G, WiFi, and poor connectivity conditions
- **API Load Testing:** Stress testing for Affiliated Monitoring API integrations
- **Battery Impact:** Analysis and optimization of mobile battery consumption

Regression Testing Suite

- **Critical User Flows:** Automated regression testing for authentication, device monitoring, emergency contacts, billing
- **API Integration Testing:** Continuous validation of Affiliated Monitoring API integrations
- **Cross-Platform Consistency:** Ensuring parity across iOS, Android, and web
- **Data Integrity Testing:** Validation of data synchronization across platforms

Manual Testing & User Acceptance

- **Exploratory Testing:** Manual edge-case validation and scenario-based testing
- **Usability Testing:** Real user testing with seniors and caregivers
- **Emergency Scenario Testing:** Validation of critical emergency response workflows and notifications
- **Accessibility Manual Testing:** Human testing with assistive technology users

Release Readiness Checklist

Pre-Release Validation

- Performance benchmarks met (startup time, memory usage, API response times)
- Security scan completed with no critical vulnerabilities
- Cross-platform feature parity confirmed
- App Store guidelines compliance verified
- HIPAA compliance documentation complete

Deployment Validation

- Staging environment testing with production-like data
- Third-party service integrations validated (Twilio, Stripe, Google Maps)
- Push notification delivery testing across devices
- Database migration and data integrity verification
- Monitoring and alerting systems operational
- Rollback procedures tested and documented

Quality Metrics & Reporting

- **Bug Tracking:** Defect lifecycle management in Jira with severity classification

- **Performance Metrics:** Continuous monitoring with automated alerts for regressions
- **User Experience Metrics:** Post-launch monitoring of engagement, feature adoption, and support tickets
- **Accessibility Compliance:** Regular audits with detailed reporting

Risk Mitigation & Contingency Planning

- **Critical Bug Response:** 4-hour response for issues affecting senior safety or emergency response
- **Rollback Strategy:** Immediate rollback capability with data integrity preservation
- **Emergency Support:** 24/7 support during initial launch with escalation procedures
- **Performance Monitoring:** Real-time monitoring with automated alerting for degradation

16: Development Process & Tools Methodology

Code Management & Version Control

- GitHub as primary repository with branch protection, pull request reviews.
- **Branching Strategy:** GitFlow – feature branches, development branch, protected main branch
- All code changes require **peer review, ai review and automated testing** before merge

Project Management & Scrum Implementation

- **Sprint Planning:** Story point estimation, capacity planning, sprint goal definition
- **Bug Tracking:** Automated bug lifecycle with priority classification
- **Feature Status:** Real-time tracking with stakeholder visibility
- **Backlog Management:** Product backlog prioritization with LifeStation stakeholder input

Quality Assurance & Testing Strategy

- **Playwright:** Cross-browser web testing with automated UI validation
- **Detox:** React Native end-to-end testing for iOS/Android
- **Manual Testing:** Device-specific testing on physical devices

- **Accessibility Testing**

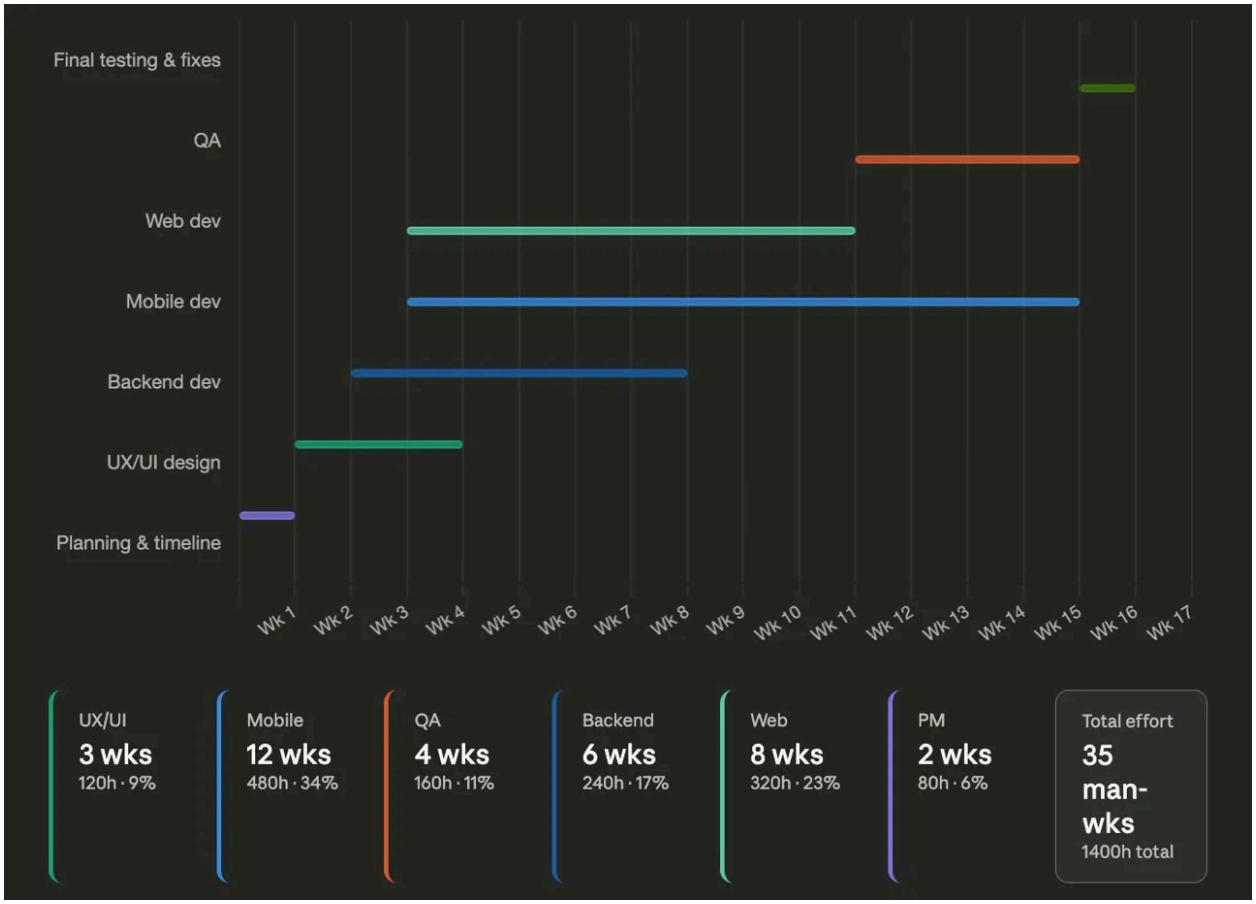
Communication & Collaboration Tools

- **Jira Integration:** Slack/Teams notifications for sprint updates and bug reports
- **Daily Standups:** Video calls with screen sharing for technical discussions
- **Sprint Reviews:** Live demonstrations of completed features
- **Documentation:** Maintained in GitHub Wiki or Confluence

Why These Tools & Processes

- **GitHub:** Industry standard for nodejs, React, React Native
- **Jira:** Transparency and real-time visibility for LifeStation team
- **Playwright & Detox:** Ensures reliable testing across platforms
- **Agile/Scrum:** Rapid iteration and stakeholder feedback
- **External Tool Coordination:** Seamless integration with LifeStation team

17. Timeline & Estimates



Full project (16 calendar weeks, 35 man-weeks / 1,400 hours)

Phases run as follows, with several tracks overlapping in parallel:

- **Planning & timeline** — Week 1 (1 wk)
- **UX/UI design** — Weeks 2–4 (3 wks)
- **Backend dev** — Weeks 3–8 (6 wks)
- **Mobile dev** — Weeks 5–16 (12 wks) ← starts same week as Web
- **Web dev** — Weeks 5–12 (8 wks) ← starts same week as Mobile
- **QA** — Weeks 14–16 (4 wks)
- **PM** — Weeks 1–16: partial (2 wks)

Mobile-only scenario (~13 calendar weeks, ~25 man-weeks / ~920 hours)

Dropping web removes 8 man-weeks of dev effort, lightens QA and PM, and accelerates calendar delivery by roughly **3 weeks**.

18. Staffing Model

- Project Manager
- UX Designer
- React Native Engineer
- Backend Engineer
- QA
- DevOps

Total: 6 people

19. Risks & Mitigation

- Billing API readiness
- UX expectations
- HIPAA compliance

Mitigation: early validation, prototypes, security baseline

20. Post-Launch Support

- 30–60 days stabilization
- Ongoing support

21. Case Study – LifeStation v1

- React Native app
- API integrations
- Device monitoring
- Notifications
- Geofencing

Advantage: deep system familiarity

22. Work-for-Hire

All deliverables owned by LifeStation

23. Proposal Submission Summary (LifeStation Caregiver App v1.0)

23.1. Executive Summary

- Delivering v1.0 mobile app (iOS/Android) + web platform.
- Core features: SMS authentication, real-time device monitoring, emergency contacts, secure billing for D2C accounts.
- Enhanced features: GPS geofencing with interactive setup, in-app calling, medication reminders with push notifications.
- Supports multiple user roles: Admin, Family Members, Caregivers, and Seniors.
- Scalable platform for evolving family ecosystems and LSaaS strategy.

23.2. Relevant Case Studies

- Experience with IoT-connected PERS devices and senior care monitoring.
- Expertise in healthcare communications, real-time alerts, and senior-accessible UI.
- Skilled in React Native, API integrations, and reliability for healthcare apps.
- **Proven LifeStation experience:** We have already delivered a version of the LifeStation caregiver app, giving us deep, hands-on knowledge of the APIs, data flows, and integration patterns.

23.3. Technical Architecture

- React Native for mobile development with dedicated React web application.
- New Architecture: TurboModules and Fabric for enhanced performance.
- State management: Zustand; data persistence via AsyncStorage.
- Backend: Node.js/Express + PostgreSQL database.
- Authentication: JWT with refresh token rotation; encrypted API integrations.
- Real-time processing: WebSocket connections for device events and notifications.
- HIPAA-compliant security: end-to-end encryption, audit logs, role-based access control.

23.4. UX & Discovery Methodology

- Collaborative workshops with LifeStation stakeholders; contextual interviews with caregivers and seniors.
- Competitive analysis: Medical Guardian, Freeus, Aloe Care apps.
- Figma interactive prototypes for all user flows; structured user testing.
- Accessibility-first design: large touch targets, high contrast, readable fonts.
- Optimized workflows: 1-tap location visibility, 2-tap emergency calls.

23.5. Staffing Model

- [To be provided: onshore/offshore resource allocation and percentages]

23.6. Role-by-Role Hourly Rates

- [To be provided: rates for Solution Architect, Backend/Frontend Engineers, UX/UI Designer, QA, DevOps, Project Manager]

23.7. Blended Rate Calculation

- [To be provided: based on resource mix & hourly rates]

23.8. Development Methodology

- Agile delivery with 2-week sprints, daily standups, and weekly status reviews.
- Sprint planning, story point estimation, and clear acceptance criteria.
- CI/CD pipelines for automated testing and deployment.
- Shared Jira backlog for project visibility and progress tracking.
- Iterative delivery with rapid feedback incorporation.

23.9. QA / Testing Approach

- Automated testing: unit (>90% coverage), integration, end-to-end (Detox).
- Device coverage: iPhone 12–17, Samsung Galaxy S21+, Pixel 6+, OnePlus 9+.
- Accessibility testing.
- Performance: app startup <3 sec; optimized memory usage; API load testing.
- Security testing: penetration tests, HIPAA compliance validation.

23.10. Risk Assessment

- Technical risks: third-party API performance, React Native limitations, HIPAA compliance complexity.
- Project risks: scope creep, app store delays, resource availability.

- Business risks: competitive feature gaps, mitigated through extensible architecture and continuous analysis.

23.11. Timeline / Milestones

- TBD - Detailed project timeline and milestones to be finalized upon receipt of functional specifications from LifeStation, expected within 2 weeks of project start as noted in RFP

23.12. Post-Launch Support

- 60-day stabilization support with 24/7 critical issue response (4-hour SLA).
- Daily monitoring, performance reports, bug fixes, and hotfix deployment.
- Ongoing support options: monthly retainer, quarterly health checks, sprint-based support.
- Future Phase 2 potential: advanced geofencing, activity telemetry, Bluetooth connectivity, AI-powered anomaly detection.

23.13. Assumptions & Exclusions

Assumptions

- LifeStation provides timely staging/development access and project feedback
- Firebase project setup for push notifications provided by LifeStation
- Functional specifications finalized within 2 weeks of project start
- LifeStation serves as publisher for Apple App Store & Google Play Store
- Customer database API access provided for account validation
- Weekly coordination meetings with Dan Coppola and technical team

AIAssistant.co Will Provide (Included in Scope)

- **SMS Service:** Twilio integration and configuration
- **Payment Processing:** Stripe integration for secure billing
- **Maps Integration:** Google Maps setup for location/geofencing
- **Development Tools:** GitHub, Jira, testing frameworks
- **QA Tools:** Playwright

Exclusions

- HIPAA legal consulting and regulatory audits
- Production infrastructure provisioning (AWS provided by LifeStation)

- Customer support training/documentation for LifeStation staff
- Marketing materials, App Store optimization, and promotional content
- Integration with systems not specified in RFP
- Data migration from existing systems or legacy platforms
- Third-party vendor contract negotiations beyond technical integration

External Service Costs (Estimated Monthly)

- **Twilio SMS:** ~$200–500/month
- **Stripe Processing:** 2.9% + 30¢ per transaction
- **Google Maps Platform:** ~$200–800/month
- **Development Tools:** GitHub Team (~$4/user/month), Jira (~$7/user/month)
- **Design Tools**: Figma Professional (~$12/user/month for design team)
- **Testing Services:** AWS Device Farm / BrowserStack (~$100–300/month)
- **Analytics & Monitoring:** Covered by LifeStation (Mixpanel, Datadog, Crashlytics, Userpilot)

Note: Costs are estimates and depend on actual usage volumes. Final costs should be validated during implementation planning.

24. Conclusion

AIAssistant.co has extensive experience with LifeStation app development and API integration. We propose to deliver a complete solution aligned with RFP requirements, focusing on UX, security, and scalability.

We look forward to the opportunity to deliver a standout caregiver experience.

Primary Contact:
[Vaidhi: Please provide your Name, Title, Email, and Phone here]

Document prepared by AIAssistant.co — March 2026